

Les exceptions dans MYSQL

Les exceptions dans MYSQL

Introduction à la gestion des erreurs dans MYSQL:

Il est très important de gérer les erreurs dans les programmes stockés MySQL tels que les procédures, les fonctions... Si vous ne gérez pas les erreurs de manière appropriée, l'utilisateur final verra un message d'erreur généré par le système qui n'est pas compréhensible pour tout le monde.

Une exception est détectée (aussi dite "levée") si elle est prévue dans un **handler** au cours de l'exécution d'un bloc (entre **BEGIN** et **END**).

Deux mécanismes qui peuvent être mis en œuvre pour gérer une exception en MYSQL : **CONTINUE** et **EXIT**.

Déclarer un gestionnaire d'erreurs :

Nous pouvons déclarer un gestionnaire d'erreurs à l'aide de la syntaxe suivante :

```
DECLARE handler_action HANDLER
  FOR condition_value [, condition_value] ... statement;
```

- Pour **handler_action** : { **CONTINUE** | **EXIT** | **UNDO** }
- Pour **condition_value** : { **mysql_error_code** | **SQLSTATE sqlstate_code** | **condition_name** | **SQLWARNING** | **NOT FOUND** | **SQLEXCEPTION** }

La syntaxe générale :

```
DECLARE { CONTINUE | EXIT } HANDLER
  FOR { mysql_error_code | SQLSTATE sqlstate_code | condition_name | NOT FOUND }
  statement;
```

Ici, on peut spécifier l'une des trois actions de gestionnaire suivante :

- **CONTINUE** : Le programme actuel continue son exécution.
- **EXIT** : L'instruction composée **BEGIN... END** dans laquelle le gestionnaire est déclaré termine l'exécution.
- **UNDO** : MYSQL ne le supporte pas.

Les conditions spécifiques qui activent le gestionnaire sont spécifiées par :

- **mysql_error_code** : Un code d'erreur MySQL représenté par un entier, tel que **1146** pour "table inexistante".
- **SQLSTATE sqlstate_code** : une chaîne lde 5 caractères spécifiant une valeur SQLSTATE, telle que '**42S02**' pour désigner une "table inexistante".
- **condition_name** : le nom de condition précédemment défini à l'aide de : **DECLARE condition_name CONDITION FOR condition_value**. Un nom de condition peut être connecté à une valeur **SQLSTATE** ou à un numéro d'erreur MySQL. Par exemple : **DECLARE dividedByZero CONDITION FOR SQLSTATE '22012'**;

Pour plus d'informations sur les codes erreurs dans MYSQL, consulter le lien :

<https://dev.mysql.com/doc/mysql-errors/8.0/en/server-error-reference.html>

Exemple 1 : exception avec EXIT avec code erreur

Nous allons créer la table **employees** suivante :

```
CREATE TABLE employees(  
id INT PRIMARY KEY,  
dept VARCHAR(10),  
city VARCHAR(20)  
);
```

Nous allons insérer un enregistrement dans la table **employees** à l'aide de l'instruction **INSERT** suivante :

```
INSERT INTO employees VALUES(null,'IT','AGADIR');
```

Une exception est déclenchée ayant le code d'erreur **1048** :

12 16:47:29 INSERT INTO employees VALUES(null,'IT','AGADIR')	Error Code: 1048. Column 'id' cannot be null
--	--

On va essayer de créer une procédure stockée pour gérer cette exception à l'aide du **HANDLER EXIT** :

```
DELIMITER //  
CREATE PROCEDURE insert_employee1(IN id INT, IN dept VARCHAR(10), IN city  
VARCHAR(20))  
BEGIN  
    DECLARE is_null BOOLEAN DEFAULT FALSE;  
    BEGIN # Début du bloc handler  
        DECLARE EXIT HANDLER FOR 1048 SET is_null = TRUE;  
        INSERT INTO employees VALUES(id, dept, city);  
        SELECT 'Ajout de l\'employé avec succès' AS MESSAGE;  
    END; # Fin du block handler  
    IF is_null = TRUE THEN  
        SELECT 'Le champ Id ne doit pas être null' AS ErrorMessage;  
    END IF;  
END //  
DELIMITER ;  
  
CALL insert_employee1(1, 'IT', 'AGADIR');
```

MESSAGE
Ajout de l'employé avec succès

```
CALL insert_employee1(NULL, 'IT', 'AGADIR');
```

ErrorMessage
Le champ Id ne doit pas être null

Exemple 2 : exception avec EXIT avec SQLSTATE

Dans la procédure stockée, nous allons créer un gestionnaire d'erreurs de sortie pour **SQLSTATE : 23000** (le code d'erreur 1062) qui concerne les entrées en double dans la table **employees** :

```
DELIMITER //
CREATE PROCEDURE insert_employee2(IN id INT, IN dept VARCHAR(10), IN city
VARCHAR(20))
BEGIN
    DECLARE is_duplicated BOOLEAN DEFAULT FALSE;
    BEGIN # Début du bloc handler
        DECLARE EXIT HANDLER FOR SQLSTATE '23000' SET is_duplicated = TRUE;
        INSERT INTO employees VALUES(id, dept, city);
        SELECT 'Ajout de l\'employé avec succès' AS MESSAGE;
    END; # Fin du block handler
    IF is_duplicated = TRUE THEN
        SELECT CONCAT('L\'entrée en double pour l\'identifiant : ',id) AS
        ErrorMessage;
    END IF;
END //
DELIMITER ;

CALL insert_employee2(2,'IT','AGADIR');
```

MESSAGE
Ajout de l'employé avec succès

```
CALL insert_employee2(1,'IT','AGADIR');
```

ErrorMessage
L'entrée en double pour l'identifiant : 1

Exemple 3 : exception avec EXIT avec NOT FOUND

Dans la procédure stockée, nous allons créer un gestionnaire d'erreurs de sortie pour **NOT FOUND** qui concerne un employé qui n'existe pas dans la table **employees** :

```
DELIMITER //
CREATE PROCEDURE search_employee(IN id_emp INT)
BEGIN
    DECLARE is_present BOOLEAN DEFAULT TRUE;
    DECLARE dept_var VARCHAR(10);
    DECLARE city_var VARCHAR(20);
    BEGIN # Début du bloc handler
        DECLARE EXIT HANDLER FOR NOT FOUND SET is_present = FALSE;
        SELECT dept, city INTO dept_var, city_var FROM employees WHERE id=id_emp;
        SELECT id_emp, dept_var, city_var;
    END; # Fin du block handler
    IF is_present = FALSE THEN
        SELECT 'Employé non trouvé' AS ErrorMessage;
    END IF;
END //
DELIMITER ;

CALL search_employee(1);
```

	id_emp	dept_var	city_var
▶	1	IT	AGADIR

```
CALL search_employee(10);
```

	ErrorMessage
▶	Employé non trouvé

Exemple 4 : exception avec CONTINUE

L'exemple suivant décrit une procédure qui gère une erreur SQLSTATE : '23000' causé par la duplication de la clé primaire :

```
CREATE TABLE codes(id INT PRIMARY KEY);

DELIMITER //
CREATE PROCEDURE proc()
BEGIN
    DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET @Y=2;
    SET @X=1;
    INSERT INTO codes VALUES(1);
    SET @X=2;
    INSERT INTO codes VALUES(1);
    SET @X=3;
END //
DELIMITER ;

SET @X=0;
SET @Y=0;
SELECT @X, @Y;
```

	@X	@Y
▶	0	0

```
CALL proc();
SELECT @X, @Y;
```

	@X	@Y
▶	3	2

L'instruction SIGNAL

L'instruction **SIGNAL** permet de renvoyer une condition d'erreur ou d'avertissement à partir d'un sous-programme.

Voici la syntaxe d'utilisation de SIGNAL :

```
SIGNAL SQLSTATE | condition_name;
SET condition_information_item_name_1 = value_1,
    condition_information_item_name_2 = value_2, ...;
```

Dans l'exemple suivant la procédure stockée suivante ajoute un élément dans la **table2** mais elle émet un message d'erreur si le code1 n'existe pas dans la **table1** :

```
CREATE TABLE table1(code1 INT PRIMARY KEY);
INSERT INTO table1 VALUES(1);
INSERT INTO table1 VALUES(2);
INSERT INTO table1 VALUES(3);
CREATE TABLE table2(code2 INT PRIMARY KEY, code1 INT);

DELIMITER //
CREATE PROCEDURE my_proc(c2 INT, c1 INT)
BEGIN
    DECLARE c INT;
    SELECT COUNT(code1) INTO c FROM table1 WHERE code1=c1;
    IF c=0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='CODE INCORRECT !';
    END IF;
    INSERT INTO table2 VALUES(c2,c1);
END //
DELIMITER ;

CALL my_proc(1,1);
SELECT * FROM table2;
```

	code2	code1
▶	1	1
*	NULL	NULL

```
CALL my_proc(2,4);
```

✖ 87 20:16:04 CALL my_proc(2,4)

Error Code: 1644. CODE INCORRECT !

L'instruction RESIGNAL

L'instruction **RESIGNAL** est similaire à l'instruction **SIGNAL** en termes de fonctionnalité et de syntaxe sauf que l'instruction RESIGNAL doit être utilisé dans un gestionnaire d'erreurs.

Dans l'exemple suivant la procédure stockée accepte deux nombres et effectue une opération de division si le diviseur est 0, cela affiche une erreur :

```
DELIMITER //
CREATE PROCEDURE sample(IN dividend INT, IN divisor INT, OUT res DOUBLE)
BEGIN
    DECLARE dividedByZero CONDITION FOR SQLSTATE '22012';
    DECLARE CONTINUE HANDLER FOR dividedByZero RESIGNAL SET MESSAGE_TEXT = 'Error
    Denominator given is zero';
    IF divisor=0 THEN
        SIGNAL dividedByZero;
    ELSE
        SET res = dividend / divisor;
    END IF;
END //
DELIMITER ;

CALL sample(1,2,@res);
SELECT @res;
```

	@res
▶	0.5

```
CALL sample(1,0,@res);
```



94 20:44:12 CALL sample(1,0,@res)

Error Code: 1644. Error Denominator given is zero